Tensor Calculus
03/30/2026

Tensor calculus emerges as the natural extension of vector calculus when one seeks to formulate physical laws in a form that remains valid under arbitrary coordinate transformations. This article begins with a review of vector calculus, then builds the foundational concepts of tensor calculus, including its algebraic and differential structures, and concludes with applications in linear algebra, neural networks, and continuum mechanics.

Vector Calculus

Vector calculus operates on *Euclidean* space $\mathbb{R}^3$ (or $\mathbb{R}^n$) with a fixed Cartesian coordinate system. A scalar field $f(\mathbf{x})$ assigns a scalar to each point, while a vector field $\mathbf{v}(\mathbf{x})$ assigns a vector. The fundamental differential operations are defined in a coordinate-dependent manner but are often expressed in a coordinate-free vector notation.

In Cartesian coordinates with orthonormal basis $\{\mathbf{e}_i\}$, a vector field is written as $\mathbf{v} = v^i \mathbf{e}_i$, where the components v^i are functions of the coordinates x^i. The gradient of a scalar field is

$$\nabla f = \frac{\partial f}{\partial x^i} \mathbf{e}^i,$$

where $\mathbf{e}^i$ are the dual basis vectors (in Cartesian coordinates, $\mathbf{e}^i = \mathbf{e}_i$). In component form, $(\nabla f)_i = \partial_i f \text{ with } \partial_i \equiv \frac{\partial}{\partial x^i}$. The divergence of a vector field is

$$\nabla \cdot \mathbf{v} = \frac{\partial v^i}{\partial x^i},$$

and the curl in three dimensions is

$$\nabla \times \mathbf{v} = \epsilon^{ijk} \frac{\partial v_k}{\partial x^j} \mathbf{e}_i,$$

where ϵ^{ijk} is the *Levi-Civita* symbol. The *Laplacian* of a scalar field is $\nabla^2 f = \partial_i \partial^i f$. These operations are sufficient for many problems in physics and engineering, yet they rely implicitly on the existence of a global Cartesian coordinate system and a *Euclidean* metric. When working with curvilinear coordinates or curved spaces, a more flexible framework is required—this is the domain of tensor calculus.

Tensor Calculus

Tensor calculus is built on the principle of *coordinate invariance*. A geometric or physical quantity is represented by a tensor, whose components transform in a prescribed way under a change of coordinates, ensuring that the quantity itself is independent of the coordinate system. The manifold on which tensors live is assumed to be differentiable, allowing the definition of tangent vectors and covectors (linear maps that take a vector as input and produces a scalar as output) at each point. A tensor of type (r, s) (contravariant of rank r, covariant of rank s) is defined by its components $T^{i_1 \ldots i_r}{}_{j_1 \ldots j_s}$ that transform as

© All rights reserved, Orange County Science Institute, 2026

$$T'^{i'_1 \ldots i'_r}{}_{j'_1 \ldots j'_s} = \frac{\partial x'^{i'_1}}{\partial x^{i_1}} \cdots \frac{\partial x'^{i'_r}}{\partial x^{i_r}} \frac{\partial x^{j_1}}{\partial x'^{j'_1}} \cdots \frac{\partial x^{j_s}}{\partial x'^{j'_s}} T^{i_1 \ldots i_r}{}_{j_1 \ldots j_s}$$

The *Einstein* summation convention is employed: repeated indices, one upper and one lower, are summed over. The metric tensor g_{ij} is a symmetric, non-degenerate covariant tensor of rank 2 that defines inner products and distances. Its inverse g^{ij} satisfies $g^{ik} g_{kj} = \delta^i_j$.

Addition and Product Rules

Tensors of the same type form a vector space. Their addition is componentwise:

$$(T + S)^{i_1 \ldots i_r}{}_{j_1 \ldots j_s} = T^{i_1 \ldots i_r}{}_{j_1 \ldots j_s} + S^{i_1 \ldots i_r}{}_{j_1 \ldots j_s}$$

The *tensor product* (outer product) combines a tensor T of type (r, s) and U of type (p, q) into a tensor $T \otimes U$ of type $(r + p, s + q)$:

$$(T \otimes U)^{i_1 \ldots i_r \, k_1 \ldots k_p}{}_{j_1 \ldots j_s \, l_1 \ldots l_q} = T^{i_1 \ldots i_r}{}_{j_1 \ldots j_s} \, U^{k_1 \ldots k_p}{}_{l_1 \ldots l_q}.$$

Contraction reduces the rank by setting a contravariant index equal to a covariant index and summing. For example, a type $(1, 1)$ tensor T^i_j contracts to the scalar T^i_i. In the presence of a metric, indices can be raised or lowered:

$$T^i_j = g^{ik} T_{kj}, \qquad T_{ij} = g_{ik} g_{jl} T^{kl}$$

allowing the conversion between covariant and contravariant components. This operation also defines the inner product of vectors: $\mathbf{u} \cdot \mathbf{v} = g_{ij} u^i v^j$.

Derivative Rules

The ordinary partial derivative of a tensor field does not generally transform as a tensor under coordinate changes. To obtain a derivative that respects the tensor character, one introduces the *covariant derivative* ∇. For a vector field v^i, the covariant derivative is

$$\nabla_j v^i = \partial_j v^i + \Gamma^i{}_{jk} v^k$$

where $\Gamma^i{}_{jk}$ are the *Christoffel* symbols (connection coefficients) that encode how basis vectors change across the manifold. For a covector ω_i,

$$\nabla_j \omega_i = \partial_j \omega_i - \Gamma^k{}_{ji} \omega_k$$

The extension to arbitrary tensors follows the *Leibniz* rule:

$$\nabla_k (T \otimes U) = (\nabla_k T) \otimes U + T \otimes (\nabla_k U)$$

© All rights reserved, Orange County Science Institute, 2026

and linearity. A fundamental property of the *Levi-Civita* connection (the unique torsion-free connection that is metric-compatible) is $\nabla_k g_{ij} = 0$. This ensures that raising and lowering indices commute with covariant differentiation.

The covariant derivative reduces to the ordinary partial derivative in Cartesian coordinates, where ${\Gamma^i}_{jk} = 0$. In curvilinear coordinates, it provides the correct expressions for gradient, divergence, and curl in a coordinate-invariant manner. For instance, the divergence of a vector field is given by

$$\nabla_i v^i = \frac{1}{\sqrt{g}} \frac{\partial}{\partial x^i} \left(\sqrt{g}\, v^i \right)$$

where $g = \det(g_{ij})$. The *Laplacian* of a scalar field becomes

$$\nabla^2 f = \nabla_i \nabla^i f = \frac{1}{\sqrt{g}} \frac{\partial}{\partial x^i} \left(\sqrt{g}\, g^{ij} \frac{\partial f}{\partial x^j} \right)$$

<u>Applications in Linear Algebra</u>

Linear algebra is essentially the study of tensors of rank at most two. A vector is a type $(1,0)$ tensor, a covector a type $(0,1)$ tensor, and a linear transformation a type $(1,1)$ tensor. Matrix multiplication is a contraction: given matrices ${A^i}_j$ and ${B^j}_k$, their product is ${C^i}_k = {A^i}_j {B^j}_k$. The trace of a matrix is the contraction of its $(1,1)$ representation, $\mathrm{tr}(A) = {A^i}_i$.

Eigenvalue problems are invariant statements: for a symmetric bilinear form g_{ij} and a linear operator ${A^i}_j$, one seeks vectors v^i such that ${A^i}_j v^j = \lambda v^i$. The metric is used to raise and lower indices when necessary, *e.g.*, to define the *Rayleigh* quotient $R = (v^i g_{ij} v^j)^{-1} (v^i g_{ij} {A^j}_k v^k)$.

Higher-order tensors generalize matrices to multi-way arrays. The *singular value decomposition* (SVD) of a matrix $M = U \Sigma V^\top$ extends to the *Tucker decomposition* for a third-order tensor $\mathcal{T} \in \mathbb{R}^{I \times J \times K}$:

$$\mathcal{T} = \mathcal{G} \times_1 \mathbf{U} \times_2 \mathbf{V} \times_3 \mathbf{W},$$

where $\mathcal{G}$ is the core tensor (analogous to Σ), and $\times_n$ denotes mode-n multiplication. The *CANDECOMP/PARAFAC* (CP) decomposition writes $\mathcal{T}$ as a sum of rank-1 tensors:

$$\mathcal{T} = \sum_{r=1}^{R} \lambda_r \, \mathbf{u}_r \otimes \mathbf{v}_r \otimes \mathbf{w}_r$$

These decompositions are fundamental in machine learning for dimensionality reduction, latent factor analysis, and representation learning, and they rely entirely on the multilinear operations defined in tensor calculus.

<u>Applications in Neural Networks</u>

Neural networks operate on multi-dimensional arrays that are naturally represented as tensors. In a fully connected layer, the input $\mathbf{x} \in \mathbb{R}^n$ and weight matrix $\mathbf{W} \in \mathbb{R}^{m \times n}$ produce an output $\mathbf{y} = \mathbf{W}\mathbf{x}$. In

© All rights reserved, Orange County Science Institute, 2026

component form, $y^i = W^i_j x^j$, which is a contraction of a type $(1,1)$ tensor with a type $(1,0)$ tensor. Convolutional layers perform a more structured contraction: a kernel K of shape $(c_{\text{in}}, k_h, k_w)$ convolved with an input feature map F of shape (c_{in}, h, w) yields an output O via

$$O^i_{x,y} = \sum_{j=1}^{c_{\text{in}}} \sum_{u=1}^{k_h} \sum_{v=1}^{k_w} K^i_{j,u,v} \, F^j_{x+u-1,y+v-1}$$

which is a multilinear operation combining contraction and summation over spatial dimensions.

Backpropagation computes gradients of a loss function $\mathcal{L}$ with respect to parameters, which are themselves tensors. The chain rule in tensor notation expresses the gradient as

$$\frac{\partial \mathcal{L}}{\partial W^i_j} = \frac{\partial \mathcal{L}}{\partial y^k} \frac{\partial y^k}{\partial W^i_j}$$

Given that $y^k = W^k_l x^l$, we have $\partial y^k / \partial W^i_j = \delta^k_i x^j$, leading to $\partial \mathcal{L} / \partial W^i_j = (\partial \mathcal{L} / \partial y^i) x^j$, which is an outer product of the error vector and the input vector. Modern automatic differentiation systems implement these tensor operations efficiently, treating every layer as a differentiable tensor function.

Beyond simple feedforward networks, tensor decompositions are used to compress large models. For instance, a weight tensor of a convolutional layer can be approximated by a Tucker decomposition or a CP decomposition, reducing the number of parameters while preserving accuracy. This leverages the same multilinear algebra that forms the basis of tensor calculus.

<u>Applications in Continuum Mechanics</u>

Continuum mechanics is a field where tensor calculus is indispensable. The state of deformation is described by the *deformation gradient tensor* $\mathbf{F} = \dfrac{\partial \mathbf{x}}{\partial \mathbf{X}}$, with components $F^i{}_A = \dfrac{\partial x^i}{\partial X^A}$, where X^A are material coordinates and x^i are spatial coordinates. This two-point tensor transforms covariantly in the material indices and contravariantly in the spatial indices.

Stress is represented by the *Cauchy stress tensor* $\boldsymbol{\sigma}$, a symmetric type $(2,0)$ tensor whose components σ^{ij} give the force per unit area in the spatial configuration. The balance of linear momentum in the current configuration is expressed in vector form as

$$\nabla \cdot \boldsymbol{\sigma} + \rho \mathbf{b} = \rho \dot{\mathbf{v}}$$

where ρ is density, $\mathbf{b}$ body force per unit mass, and $\dot{\mathbf{v}}$ acceleration. In component form using covariant derivatives,

$$\nabla_j \sigma^{ij} + \rho b^i = \rho \dot{v}^i$$

Strain is measured by the *Green-Lagrange strain tensor* $\mathbf{E} = \dfrac{1}{2}(\mathbf{F}^\top \mathbf{F} - \mathbf{I})$, with components

$$E_{AB} = \frac{1}{2} \left(g_{ij} \frac{\partial x^i}{\partial X^A} \frac{\partial x^j}{\partial X^B} - G_{AB} \right)$$

© All rights reserved, Orange County Science Institute, 2026

where g_{ij} and G_{AB} are the spatial and material metrics, respectively. Constitutive laws relate stress to strain. For a linear elastic material, Hooke's law is $\sigma^{ij} = C^{ijkl}\varepsilon_{kl}$, where C^{ijkl} is the fourth-order elasticity tensor, possessing the symmetries $C^{ijkl} = C^{jikl} = C^{ijlk} = C^{klij}$. The tensor nature ensures that the material response is independent of the coordinate system, which is crucial for modeling anisotropic materials like crystals or composites.

In fluid mechanics, the *Navier-Stokes* equations for an incompressible *Newtonian* fluid take the vector form

$$\rho\left(\frac{\partial \mathbf{v}}{\partial t} + \mathbf{v}\cdot\nabla\mathbf{v}\right) = -\nabla p + \mu\nabla^2\mathbf{v} + \mathbf{f}$$

which in component form becomes

$$\rho\left(\frac{\partial v^i}{\partial t} + v^j\nabla_j v^i\right) = -\nabla^i p + \mu\nabla_j\nabla^j v^i + f^i$$

using the covariant derivative to correctly handle curvilinear coordinates often employed in complex geometries.

<u>Concluding Remarks</u>

Tensor calculus provides a systematic framework for expressing geometric and physical laws in a coordinate-independent manner. Its algebraic rules—addition, tensor product, contraction, and index manipulation—form a coherent language for multilinear objects. The covariant derivative extends ordinary differentiation to curved spaces and ensures that the derivative of a tensor remains a tensor. In linear algebra, the tensor viewpoint unifies vector and matrix operations and gives rise to powerful higher-order decompositions. In neural networks, tensors are the primary data structures, and the calculus of gradients is seamlessly integrated with tensor operations. In continuum mechanics, tensors capture deformation, stress, and material response, enabling the formulation of balance laws that hold in any coordinate system. Together, these applications illustrate the unifying power of tensor calculus across disciplines that demand precision, generality, and geometric insight.

© All rights reserved, Orange County Science Institute, 2026